Metis-Auto Master Plan

Nov 1, 2026 · @Mahya

How to read this plan

The biggest saving is in **retrieval**: the fees for looking up documents, plus the document chunks sent to the model. Together they are about half of all spend. The plan therefore starts with caching and context work, which can be tested mostly for free on the logs, and does routing last, because routing needs a paid quality benchmark.

- **Part 1** explains what the research found, in plain words.
- **Part 2** lists every approach for each step, why it is a good idea, and whether we build it now (core), later (bonus) or not at all (dropped).
- **Part 3** is the full to-do list per step: checks, building, testing and shipping.
- **Part 4** is the phase plan. In each core phase the three team members (A, B and C) each build a different approach to the same problem, so we can compare three versions and keep the best. Shared work, the software tools and the frontend are separate phases.
- **Part 5** lists bonus phases for the extra approaches.

Three rules apply everywhere:

1. **Metis must not add cost per request.** Rules, local models and nightly jobs are fine. An extra LLM call on every request is only allowed if it is proven to save more than it costs.
2. **About $100 in total for all API testing** (five keys of $20).
3. **Every version is measured the same way**, with the same data, prices, judge and baseline (`gpt-5.5`).

The planning prompts for each phase are in a separate doc.

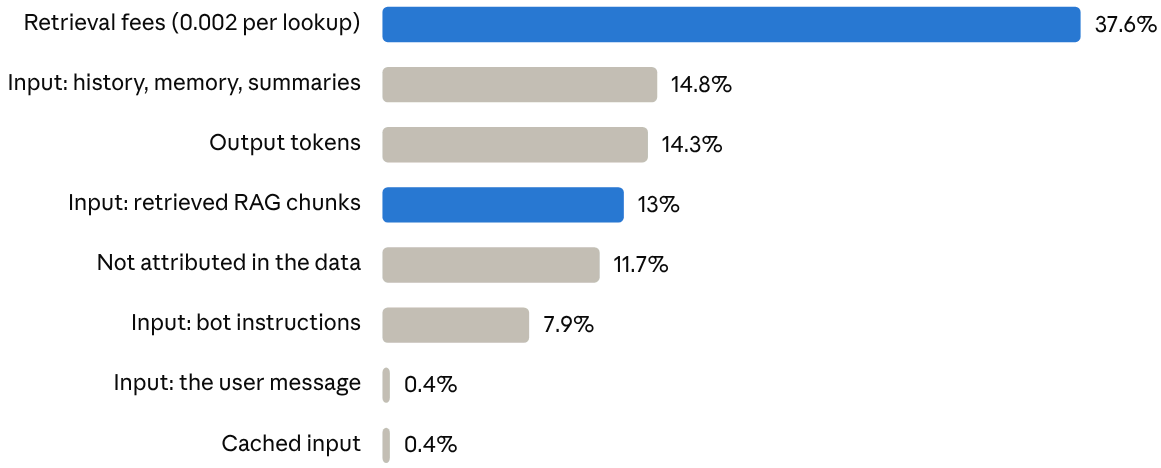
Part 1: Summary of findings

The money is mostly in retrieval, not in the instructions or the model choice. That one fact reorders the whole project.

Retrieval is about half of recorded cost

Fees plus retrieved chunks: 50.6% of recorded cost, Mar–Oct 2026

Blue = retrieval; grey = everything else. Share of recorded cost, %.



Full-dataset EDA claims ledger and the revised research reports · 80,179 sessions, Mar–Oct 2026 · input split is a regression-based reconstruction

What the data says (full EDA, 80,179 sessions)

- **Traffic is concentrated.** 5 users make 97% of requests, and 10 bots make 69%. Every bot belongs to one user and always uses one model.
- **One model carries the bill.** `gemini-2.5-flash` handles 12.5% of requests but 65% of cost.
- **30% of requests come from non-LLM workflow bots.** They cost nothing in tokens, so they must be left out of every hit rate and saving.
- **Metis already shortens history.** Stored history grows to about 34,500 characters by turn 20, but billed input only grows 1.7 times.
- **Chunks repeat a lot.** 61% of chunk rows were already sent earlier in the same session, and 33% appear twice in the same message.
- **Questions repeat, but slowly.** The median gap to a repeat is about 40 days.
- **Missing from the data:** answer quality, answers from other models, latency, failures and retries.

What each research report concluded

Report	Main conclusion in one line
Caching (revised)	Retrieval-level caching is the biggest cache lever. A popular-question cache is worth building for first turns. A semantic cache does not pay for any per-request LLM call.
Token reduction (revised)	Avoid retrieval calls first, then remove duplicate chunks, then rerank. Prompt caching is worth at most about 6–7% of cost.
RAG cost (new)	Skipping or reusing one retrieval saves about 3 times more than deleting all of that turn's chunks.
Gemini (new)	Hidden "thinking" tokens may be inflating Gemini cost. The 2.5 models are being retired, and most of our prompts are too short for Gemini's cache.
Routing (revised)	Choose the model per bot first. Per-request routing only where a bot's traffic is mixed. No LLM-based router.
Routing benchmark (new)	Logged answers cannot rank models. Build a small paired benchmark of about 500 requests for about $35, corrected with Persian human labels.

Five facts that shape the plan

1. **Cut retrieval before anything else.** It is about half of the cost and can be tested for free on the logs.
2. **Prompt caching is smaller than we thought.** The median instruction is about 370 tokens, below every provider's cache minimum.
3. **Gemini settings are a cheap check.** Turning thinking off or down, and fixing the prompt order, might cut the most expensive model's cost without any routing. A live test costs under $2.
4. **Routing needs fresh answers.** The logs show only one model per bot, so we must pay for a small benchmark.

5. **The model pool is limited to what Metis sells.** Metis's price list is about 10% above provider list prices, and some models from the research, such as Claude Haiku 5.5 and Gemini 3.8 Flash, are not on it.

Questions only Metis can answer

Several designs depend on these, so ask now (details in the last section):

- Is each 0.002 fee one retrieval call?
- Can Metis-Auto sit in front of retrieval?
- What history policy does each bot use?
- Which Gemini endpoint is used, and are cached and thinking tokens logged?
- Does the licence allow sending samples to model APIs?

Part 2: Approaches for each step

For each step we keep three competing approaches (one per person), put the safe basics that everyone needs into a shared base, keep the rest as bonus phases, and drop what breaks the no-extra-cost rule. "Core A / B / C" says who builds it.

Step 1: Caching (answers and retrieval)

Caching now covers two things: reusing a whole **answer**, and reusing a **retrieval result** (the list of chunks a lookup returned). Retrieval caching targets the 37.6% fee block, which nothing else can touch.

Approach	What it does, in plain words	Why it is a good idea	Runtime cost	Verdict
Strict exact cache	Serve a stored answer only when the bot, the message and the whole conversation so far are identical	Almost no risk; catches retries and double clicks	One Redis lookup	**Shared base** for every version

Approach	What it does, in plain words	Why it is a good idea	Runtime cost	Verdict
Popular-question cache	Every night, find the most repeated first questions per bot (last 90 days), pick the most typical past answer, have a person approve it, and serve it the next day	The top 100 questions cover 16% of first turns; 33% of sessions are single-turn; repeats are about 40 days apart, so a nightly rebuild over a long window fits	One Redis lookup	**Core A**
Retrieval-result cache	Remember which chunks a lookup returned for a normalized question, per bot and knowledge-base version, and reuse them instead of paying for the same lookup again	97% of chunk rows repeat an earlier chunk, so the same lookups keep happening; correctness can be checked for free against later real lookups in the logs	One hash lookup	**Core B**
Session retrieval memo + skip gate	Inside one conversation, reuse the previous turn's chunks for follow-up questions, and skip retrieval entirely for greetings, thanks and "say it shorter"	61% of chunk rows repeat a chunk already sent in the same session; skipping or reusing a lookup saves about 3 times more than deleting its chunks	Rules plus at most one local embedding	**Core C**

Approach	What it does, in plain words	Why it is a good idea	Runtime cost	Verdict
Semantic retrieval cache	Reuse the chunks of a past lookup whose question is very close in meaning, not just identical	More fee savings than exact matching (one study: 77% fewer database calls), but accuracy falls sharply if the threshold is loose	Local embedding + vector search	Bonus
Nightly paraphrase grouping	Offline, group questions that mean the same thing and share an answer; a person approves them; they join the popular-question cache	Adds semantic coverage with zero per-request cost	None at runtime	Bonus
Embedding and rerank cache	Store embeddings and reranker scores by text hash	Saves compute and latency; money only if these are billed	Hash lookup	Bonus (small)
Online semantic answer cache with an LLM rewrite or check	Rewrite each question with an LLM and match by meaning	Would need a 6–20% hit rate to pay for the LLM call; the measured pool is about 0.3% of requests	One LLM call per lookup	**Dropped**
Reusing the model's internal state per chunk (RAGCache, CacheBlend)	Store the model's processed state for each chunk	2–4× faster first token in papers	Needs self-hosted models	**Dropped** (we use provider APIs)

Step 2: Context (what we send to the model)

The shared base removes exact duplicate chunks and fixes the prompt order, because both are close to risk-free and every version should start from them. The three competing approaches then attack the three remaining blocks: chunks, repeated chunks across turns, and history.

Approach	What it does	Why it is a good idea	Runtime cost	Verdict
Clean context	Remove chunks that appear twice in one message (exact and after Persian normalization); put stable parts first and changing parts last; build the prompt the same way every time	Up to about 4% of total cost, near-zero risk; protects the provider caches that already work (DeepSeek 76%, OpenAI 29%)	Hashing only	**Shared base**
Reranker + relevance threshold	Score each of the 15 chunks with a local Persian-capable reranker (bge-reranker-v2-m3) and send only the useful ones, never fewer than a floor	Chunks are about 36% of input; cutting 15 to 5–8 could save about 3.5–7% of cost	About 40 ms on a GPU	**Core A**
Cross-turn chunk policy + session ledger	For a chunk already sent earlier in the session: drop it with a one-line pointer if the model can still see it, keep it if not; optionally keep session chunks in one append-only block so later turns hit the provider cache	Up to about 8% of cost on repeated chunks; the pointer method held accuracy in a 2026 study	Near zero	**Core B**

Approach	What it does	Why it is a good idea	Runtime cost	Verdict
History budget, cache-friendly	Measure each bot's current history policy, then test a tighter budget on long requests and a stepped window (drop several old turns at once, so the prompt start stays stable for caching)	Requests over 8,000 tokens are 11% of requests but 65% of input tokens; history is the largest input block (about 41%)	None	**Core C**
XProvence sentence pruning	Remove irrelevant sentences inside kept chunks, in the same pass as reranking	Supports Persian natively; extra saving on top of reranking	Same as reranking	Bonus
Explicit Gemini cache per bot	Store a bot's stable prefix (instructions plus its most-used chunks) in Gemini's paid cache	Guaranteed cache hits, but only pays for bots with about 4+ identical requests an hour	Storage fee per hour	Bonus
Offline instruction shortening	Rewrite the largest instruction templates once, offline	One-time cost; only a few % of cost because instructions are about 8%	None	Bonus (low)
LLM summaries or compression per request	Summarize or compress the prompt with an LLM on every call	Rarely pays at these prices: it must remove about 2,000 tokens per call just to break even	One LLM call	**Dropped**

Approach	What it does	Why it is a good idea	Runtime cost	Verdict
LLMLingua-style token pruning at strong rates	Delete "unimportant" words with a model trained on English	Loses much more meaning outside English	Encoder pass	**Dropped** (except mild rates, bonus)
Memory layers (Mem0, Zep)	External memory systems	Built for long multi-session memory; our sessions are short (median 2 user messages)	Extra LLM calls	**Dropped**

Step 3: Routing (which model answers)

All three routers learn from the same paid benchmark. The first decision is per bot, because the model is already a bot setting.

Approach	What it does	Why it is a good idea	Runtime cost	Verdict
Shared benchmark	About 500 logged requests, each answered again by 4–5 models on the same rebuilt prompt, graded by two cheap judges and corrected with about 250 Persian human labels	The logs cannot say which model is better; this is the smallest fair test	Development only (about $35)	**Shared base**

Approach	What it does	Why it is a good idea	Runtime cost	Verdict
Per-bot reassignment + rules	Move each bot to the cheapest model that is good enough for it; add simple rules (images need a vision model, very long prompts avoid price cliffs, retiring models are replaced)	Zero runtime cost, matches how Metis works today, and most money sits in a few bots	None (a config change)	**Core A**
kNN router	Embed the request locally, look at the most similar benchmark requests, and pick the cheapest model that did well on them	In several 2025–2026 studies kNN matched complex learned routers; adding a model only needs new labels	One local embedding, milliseconds	**Core B**
Small classifier	Logistic regression or gradient boosting on simple features (length, language, RAG, turn, structured output) plus the embedding, predicting "is the cheap model good enough?"	Gives a probability, so the cost-quality trade-off can be tuned; fast	Under 1 ms	**Core C**

Approach	What it does	Why it is a good idea	Runtime cost	Verdict
Cascade with non-LLM checks	Answer with the cheap model, check it with rules (valid JSON, right language, not cut off), escalate if it fails	The only way to fix a bad cheap answer, but escalations pay and wait twice	Extra call only on failure	Bonus (verifiable outputs only)
Gemini model migration	Benchmark replacements for the retiring 2.5 models	`gemini-2.5-flash` is 65% of cost and limited to existing users	None	Bonus (urgent)
Fine-tuned encoder router	Train a small multilingual model on labels	Needs thousands of labels we cannot afford; kNN does as well	Low	**Dropped**
LLM or API router, commercial routers	Ask an LLM or a vendor which model to use	Adds an LLM call per request; commercial routers did not beat the best single model in benchmarks	One LLM call or a vendor hop	**Dropped**
Learning a router from logged answers alone	Compare logged answers across bots	Each bot always used one model, so this measures the tenant, not the model	—	**Dropped**

Step 0: Free quick checks (not an approach, but they change the numbers)

- **Gemini thinking:** compare billed output tokens with answer length on `gemini-2.5-flash` rows. If they are much larger, hidden thinking is being billed, and one setting could cut that cost.

- **Gemini caching:** check whether any Google row logs cached tokens; if none do, it may be a logging gap.
- **Retrieval:** confirm that one fee equals one lookup, and whether the logged retrieval query differs from the user message (a hidden rewriter).

Part 3: Master plan, everything each step needs

Each step follows the same five stages: **check** (free measurements on the logs), **build**, **test** (free replay first, then a small paid quality check), **decide** (a rule written down before seeing results) and **ship** (behind a flag, in shadow mode first). Work happens in the existing repo; its architecture, tests and decision records stay.

Foundation (needed by every step)

Data and replay

- [ ] Merge the `full-eda` branch into `main`, so its loaders, pairing and `requests.parquet` become the shared data layer
- [ ] Build one replay table: one row per request, with bot, session, turn, language, billed tokens, cost split, retrieval query, chunk IDs and the logged answer
- [ ] Leave the 30% workflow bots out of every saving and hit rate
- [ ] Split by time: tune on March–July, report on August–October
- [ ] Rebuild prompts as they were sent: instructions, then chunks, then the last *k* history turns, then the message. Choose *k* per bot so the token count matches the billed input.

Free checks (no API cost)

- [ ] Does one 0.002 fee equal one retrieval call?
- [ ] Does the logged retrieval query differ from the user message (a hidden query rewriter)?
- [ ] Are a turn's chunks still in the next turn's prompt? This decides whether dropping repeated chunks is safe.
- [ ] What history policy does each bot use: a fixed window, a token budget, or summaries?
- [ ] `gemini-2.5-flash`: billed output tokens versus answer length (hidden thinking), and whether any Google row logs cached tokens

Shared evaluation

- [ ] Replace list prices with the Metis gateway price list in `config/reference/`, dated

- [ ] Configure the judge: one cheap judge per model family, both answer orders, strict JSON output, Persian anchor examples
- [ ] Human-label tool and protocol: two native Persian speakers, a written rubric, agreement (Cohen's κ) target of 0.6 or higher
- [ ] Prediction-powered inference (PPI, with the `ppi_py` package) to correct judge scores with the human labels
- [ ] One evaluation command per step, same metrics and baselines for every version
- [ ] Raise the live-call cap per key (for example $18 of each $20), record it in ADR-0018, and keep one shared spend record across the five keys

Admin

- [ ] Send the questions to Metis (last section)
- [ ] Confirm the licence allows sending samples to model APIs and showing them to annotators
- [ ] Rotate the Hugging Face token found in the EDA prompt

Step 1: Caching

Check

- [ ] Replay retrieval calls in time order: how many `bot + normalized query` repeat within 1, 7, 30 and 90 days?
- [ ] Session follow-ups: how often does a turn's fresh chunk set mostly overlap the previous turn's?
- [ ] Share of retrieval turns that are greetings, thanks or short meta requests
- [ ] Per bot, how often chunk text changes (knowledge-base churn), to set the freshness rules

Build

- [ ] Shared base: the cache key (bot, instruction hash, model, normalizer version, corpus version), the strict exact cache, the Persian normalizer (exists in the repo), a chronological replay simulator, and the hit-labelling tool
- [ ] A: the popular-question cache: nightly job, 90-day window (180 maximum), at least 3 occurrences in 2 sessions, most typical past answer, human approval of every served key, versioned keys in Redis, emergency blocklist

- [ ] B: the retrieval-result cache: `bot + corpus version + retrieval settings + normalized query` mapped to chunk IDs, drift detection from later real lookups, long backstop TTL
- [ ] C: the session memo and skip gate: Persian follow-up rules, a phatic word list, reuse of the previous chunk set, one local embedding at most

Test

- [ ] Free: replay every version over all logged traffic; report hits, fees avoided, tokens saved, and agreement with the chunks actually retrieved later
- [ ] Paid (about $10–15): judge a sample of served answers with a cheap judge calibrated on 400 human-labelled items
- [ ] Report every rate twice: over all requests and over LLM requests only

Decide

- [ ] Retrieval versions: retrieval agreement of 0.9 or higher with the real lookup
- [ ] Answer versions: error rate at most 1% (needs about 300 labelled hits with no errors)
- [ ] Choose the highest net saving that passes. Versions that target different costs can be combined in Phase 4.

Ship

- [ ] Behind a per-bot flag; log every lookup, hit, miss and cache version; shadow-check 1% of retrieval hits against a real lookup

Step 2: Context

Check

- [ ] Split within-message duplicates into exact copies and near-copies
- [ ] Measure formatting overhead (wrappers, role labels) by tokenizing an empty template
- [ ] Offline, on a free GPU: rerank sampled turns and plot "chunks the answer actually used" against top-k and threshold

Build

- [ ] Shared base: within-message dedup, stable prompt order, deterministic serialization
- [ ] A: the reranker with threshold and a minimum number of chunks
- [ ] B: the cross-turn policy (drop with a pointer, or keep, depending on whether the earlier copy is still visible) and the optional session ledger

- [ ] C: the calibrated history baseline, a half budget on long requests, and a stepped window

Test

- [ ] Paid ablation (about $15): 400 turns on `gemini-2.5-flash` through the batch API, with the baseline, a second baseline sample to measure noise, the shared base, and each version. A judge from another family compares each against the baseline, in both orders.
- [ ] Report by user, language and the over-8,000-token slice

Decide

- [ ] Pass if the loss rate minus the win rate stays within the noise plus 3 points, no user or language group is more than 10 points worse, and JSON validity, reply language and refusal rate do not get worse
- [ ] Pick the largest saving that passes; parts of different versions may be combined

Ship

- [ ] Ship the shared base as soon as replay confirms the saving; ship each version behind a flag

Step 3: Routing

Check

- [ ] The candidate pool = models on the Metis price list that each bot can use: incumbents `gemini-2.5-flash`, `gemini-2.5-flash-lite`, `gpt-4o-mini`, `deepseek-v4-flash`; candidates such as `gpt-6-luna`, `deepseek-flash` (V4.1), `gemini-3.5-flash-lite`, `gemini-3.1-flash-lite`; premium `gpt-5.5`
- [ ] Token counts per candidate tokenizer on Persian prompts
- [ ] Cost replay of every candidate on all logged traffic (free)

Build

- [ ] Shared base: the benchmark. 500 requests (40 pilot), quotas by bot, language, RAG, length and turn, no tenant above 35%; arms = logged answer (free), the incumbent replayed, `gpt-5.5`, and 3 cheaper candidates; two judges; 250 human labels; PPI
- [ ] Shared base: model registry, hard constraints (vision, context length, price cliffs, retirement dates), failover chain (same model elsewhere, then same tier, then one tier up)

- [ ] A: per-bot reassignment plus rules
- [ ] B: kNN on local multilingual embeddings
- [ ] C: the small classifier

Test

- [ ] Nested cross-validation grouped by session; bootstrap by bot and then session; one look at the held-out 30%
- [ ] Report savings against `gpt-5.5`, against today's logged models, and against the best single cheap model

Decide

- [ ] Move a bot to a cheaper model only if its acceptable rate is within 10 points of the incumbent (lower 95% bound, judge-corrected) and its cost per request is lower
- [ ] Keep a more complex router only if it beats per-bot reassignment outside the confidence interval

Ship

- [ ] Two weeks of shadow mode, logging what the router would have chosen, then switch bot by bot

Software tools

- [ ] Postgres for request events, routing decisions and evaluation results; Redis for the caches; Parquet and DuckDB for replay data
- [ ] MLflow for every experiment run (version, data version, prices, metrics), connected to the repo's existing run tracking (ADR-0022)
- [ ] Langfuse for traces of live calls, tagged with the pipeline version
- [ ] Prometheus and Grafana for live metrics: latency, errors, cost per minute, hit rates, escalations
- [ ] Docker Compose (`compose.yaml` exists) bringing up the whole stack; CI running lint and tests; a new decision record for each new tool

Frontend

- [ ] Extend the existing Streamlit demo (ADR-0024), or decide on a new frontend in a decision record
- [ ] Pages: an overview of savings, a savings waterfall by cost block, version comparison per step, cache monitor, routing per bot, request explorer linked to Langfuse

- [ ] Persian right-to-left text and a Persian font throughout

Final integration

- [ ] Combine the winners into one pipeline and run the final comparison: `gpt-5.5`, today's models, each step's winner alone, and the combined pipeline
- [ ] Break the saving down by step with the repo's `AccountingService.decompose`
- [ ] Write a short decision note per step: what was tried, what won, why

Part 4: Phase plan

Read a row across to see one person's full pipeline; read a column down to see the three competing versions of one step. Every core phase ends with a working product: the gateway runs with that phase's winner switched on and everything else at its default.

Each person builds one full pipeline: one cache, one context and one router

Blue boxes = competing versions (the core phases). Dashed row = shared background work.

	0 Foundation	S Tools	1 Cache	2 Context	3 Routing	4 Merge + F
Member A	Data + replay, prompt rebuild	Postgres, Redis, Compose	Popular-question cache	Reranker + threshold	Per-bot + rules	Frontend dashboard
Member B	Evaluation, judge, prices	MLflow + run tracking	Retrieval-result cache	Cross-turn chunk policy	kNN router	Combine + final eval
Member C	Free checks, Metis questions	Langfuse, Grafana	Session memo + skip gate	History budget	Small classifier	Ops, demo, load test
Shared background	Sample sets, label rubric	Benchmark pilot (40)	Benchmark main run (460)	Human labels (250 + 60)	Judge correction (PPI)	Final report

Phase plan · 3 core phases (cache, context, routing) plus foundation, tools, merge and frontend

The routing benchmark is slow (paid runs plus human labels), so it runs in the background from the tools phase onward and is ready when Phase 3 starts.

Rules that keep the versions comparable

1. **One shared base per step,** built first, before anyone starts their own version.

2. **One file per version, one owner each.** For example `routing/per_bot.py` (A), `routing/knn.py` (B), `routing/classifier.py` (C), and the same pattern for cache and context. Shared code changes only with everyone's agreement.
3. **Chosen by config.** A pipeline config names one version per step, so any combination runs without code changes.
4. **Same evaluation for everyone:** same data split, judge, prices and baselines, one command per step, results in MLflow.
5. **Write the decision rule before running.** It is in each phase below.
6. **Merge to `main` daily,** after tests pass.

Phase 0: Foundation (everyone)

Everything the three versions share. Ends with: the replay table works, the free checks are answered, and one evaluation command produces an MLflow-ready result.

Member	Builds	Done when
A	Merge `full-eda` into `main`; the replay table; prompt rebuilding with *k* per bot fitted to billed tokens	Replay table covers all LLM requests; token error per bot reported
B	Gateway price list into config; judge set-up (two families, both orders, JSON); human-label tool; PPI code; per-key spend caps and one shared spend record	`metis eval` runs on 50 items with prices, judge scores and spend
C	The free checks (fees vs lookups, hidden query rewriter, chunks carried across turns, history policy per bot, Gemini thinking and caching); sample sets for cache labels, context ablation and the routing benchmark; questions to Metis; licence check	Each check has a written answer; samples saved and versioned

Phase S: Software tools (everyone, split by tool)

Needed before the core phases, because every experiment logs to these tools. Each tool gets a short decision record that fits the repo's existing design.

Member	Builds	Done when
A	Postgres (events, routing decisions, results), Redis (caches), the Docker Compose stack, CI	`docker compose up` starts everything; tests pass in CI
B	MLflow, connected to the repo's experiment runner, logging version names, data version, price version and metrics	Every `metis eval` run appears in MLflow
C	Langfuse traces for live calls; Prometheus metrics and a Grafana board (latency, errors, cost per minute, hit rates)	A test request shows in Langfuse and on the Grafana board
Background	Benchmark pilot: 40 requests through all arms; tune the rubric and pick 8 Persian anchor examples	Rubric fixed; pilot discarded from the results

Phase 1: Cache

Shared base first (owner A, since the exact-cache experiment already exists in the repo): cache key, strict exact cache, normalizer, chronological replay and the hit-labelling tool.

Member	Builds	Done when
A	Popular-question cache: nightly job over 90 days, 3+ occurrences in 2+ sessions, most typical answer, human approval, versioned Redis keys, blocklist	Replay shows hits, saved cost and correctness; top keys reviewed
B	Retrieval-result cache: normalized query + bot + corpus version mapped to chunk IDs; drift detection; backstop TTL	Replay shows fees avoided and agreement with later real lookups
C	Session memo + skip gate: follow-up rules, phatic word list, reuse of the previous chunks	Replay shows lookups skipped or reused and agreement with the real chunks

Member	Builds	Done when
Background	Benchmark main run: 460 requests × all arms, both judges	All answers and judge scores stored

How the winner is chosen: highest net saving that passes its safety rule (retrieval agreement ≥ 0.9; answer error rate ≤ 1%). The three target different costs, so two or all three can be combined in Phase 4.

Phase 2: Context

Shared base first (owner B): within-message dedup, stable prompt order, deterministic serialization.

Member	Builds	Done when
A	Reranker + relevance threshold + minimum chunk count, tuned offline on a free GPU	Ablation result versus baseline
B	Cross-turn chunk policy (drop with pointer or keep, by visibility) and optional session ledger	Ablation result on repeat-chunk turns
C	History: calibrated baseline, half budget on long requests, stepped window	Ablation result on long, later turns
Background	Human labels: 250 comparisons plus 60 double-labelled	Agreement (κ) reported

How the winner is chosen: the largest saving whose quality loss stays within the noise plus 3 points, with no group more than 10 points worse. Parts can be combined.

Phase 3: Routing

Shared base first (owner C): model registry from the Metis price list, hard constraints, failover chain, and the corrected benchmark labels.

Member	Builds	Done when
A	Per-bot reassignment + rules	Per-bot table: chosen model, quality, cost, interval
B	kNN router on local multilingual embeddings	Cost–quality curve versus A
C	Small classifier (logistic regression or gradient boosting)	Cost–quality curve versus A

How the winner is chosen: keep the simplest router unless a more complex one beats it outside the confidence interval. A bot moves to a cheaper model only within 10 points of its current quality.

Phase 4: Merge and final evaluation, plus Phase F: Frontend

Member	Builds	Done when
A	Frontend: savings overview, savings waterfall by cost block, version comparison per step, cache monitor, routing per bot, request explorer; Persian right-to-left	Dashboard shows real results
B	Combined pipeline from the winners; final comparison against `gpt-5.5`, today's models, each winner alone, and all combined; savings broken down by step	Final report with confidence intervals
C	Production Compose profile, load test of the added latency, demo script, README	Demo runs end to end

Part 5: Bonus phases

These are the extra approaches. Each has a start condition, so it only begins when the core results say it is worth it. Bonus 1 is the exception: it is cheap and possibly large, so start it as soon as the free Gemini check is done.

#	Bonus phase	What it does	Why it might work	Start when	API cost
1	**Gemini settings and migration**	Live test of caching and thinking (7 short probes); A/B test of thinking off versus on for `gemini-2.5-flash` on about 300 items; set thinking explicitly on every Gemini call; pick a replacement for the retiring 2.5 models	`gemini-2.5-flash` is 65% of cost; hidden thinking may be a big part of it; the 2.5 models are limited to existing users	The free check shows billed output much larger than the answer text	About $2 for the probes, about $3 for the A/B test
2	Semantic retrieval cache	Reuse chunks from a past lookup whose question is very close in meaning, per bot	More fee savings than exact matching	The retrieval-result cache (B, Phase 1) passes, and replay shows at least 2 more points of fees avoided at retrieval agreement ≥ 0.9	None (replay is free)
3	Nightly paraphrase grouping	Offline, group same-meaning first questions; a person approves; they join the popular-question cache	Adds meaning-based coverage with no per-request cost	The popular-question cache (A, Phase 1) passes	None, plus review time
4	XProvence sentence pruning	Remove irrelevant sentences inside kept chunks, in the reranking pass	Extra chunk-token savings; Persian supported	The reranker (A, Phase 2) wins or passes	A small ablation (about $3)
5	Explicit Gemini cache per bot	Store a busy bot's stable prefix (instructions plus its most-used chunks, at least 2,048 tokens) in Gemini's paid cache	Guaranteed cache hits where implicit caching misses	A bot has about 4+ identical-prefix requests per hour, and the live test shows implicit caching missing	About $1 to test

#	Bonus phase	What it does	Why it might work	Start when	API cost
6	Cascade with non-LLM checks	Cheap model first; escalate when a rule check fails (invalid JSON, wrong language, cut off)	Fixes bad cheap answers on verifiable outputs	Routing benchmark shows a cheap model passes most but not all structured requests	Reuses benchmark answers
7	Multi-turn drift test	30–60 conversations where the cheap model also writes the earlier turns	Checks whether quality drops over long conversations, which the single-turn benchmark cannot show	A bot is about to move to a cheaper model	About $2
8	Offline instruction shortening	Rewrite the largest instruction templates once, keep only versions that pass	One-time cost; small but permanent saving	Phases 1–3 are done	About $2
9	Embedding and rerank cache	Store embeddings and reranker scores by text hash	Saves local compute and latency	The reranker or kNN router ships	None

Who does bonus work: whoever finishes their core version first takes the next bonus phase that is unlocked, starting with Bonus 1.

Budget, questions for Metis, risks

Budget (about $100 across five $20 keys)

Use	Phase	Estimate
Judge calibration and pilot	0, S	$5
Cache: judging served answers (400 human-calibrated items, bulk cheap judge)	1	$15
Context: 400-turn ablation with the batch API	2	$15
Routing benchmark: 500 requests, all arms, two judges	Background, 3	$35

Use	Phase	Estimate
Gemini live test and thinking A/B	Bonus 1	$5
Final comparison	4	$10
Reserve for reruns and price changes	—	$15
Total		**$100**

To stretch the budget: reuse answers across phases (the routing benchmark's incumbent replay can serve as the context baseline), use batch APIs at 50% off, and use DeepSeek off-peak as a judge where it passes calibration. Re-price every line from the Metis price list before running, since it is about 10% above provider list prices.

Questions for Metis (send in Phase 0)

1. Is each 0.002 per-call fee one retrieval lookup? In which currency is cost recorded?
2. Can Metis-Auto sit in front of retrieval calls, or only in front of the model call? (Without this, only the popular-question cache and the context work can save fees or tokens.)
3. What does each bot actually send: a history window, a summary, memory? Are earlier turns' chunks kept in later prompts?
4. Which Gemini endpoint is used (Gemini API or Vertex), native or OpenAI-compatible, and are cached and thinking tokens logged? What thinking setting do Gemini bots send?
5. Which models can each bot use? Is Claude Haiku 5.5 or Gemini 3.8 Flash available?
6. Does the licence allow sending samples to model APIs and showing them to annotators?
7. Is there a batch tier on the gateway?

Main risks

Risk	What we do about it
Metis-Auto cannot intercept retrieval calls	Cache versions B and C become "measured potential" for Metis; the popular-question cache and the context work still save money

Risk	What we do about it
Savings look bigger offline than in production	Report against today's models, not only against `gpt-5.5`; shadow mode before switching
Judges are weak in Persian	Two judge families, Persian anchor examples, 250 human labels and PPI correction; trust only strata with good agreement
A wrong cached answer is served many times	Human approval of every popular key; blocklist; per-bot off switch
`gemini-2.5-flash` disappears suddenly	Bonus 1 picks and tests a replacement early
Rebuilt prompts differ from what was really sent	Fit history length to billed tokens; compare the logged answer with the incumbent replay per bot
About 12% of cost is unexplained	Trace it before stating final savings
One tenant dominates results	No tenant above 35% of any sample; report per bot, per tenant and weighted by traffic